

Assignment 3

Team number: 22

Team members

Name	Student Nr.	Email
Darian Janevski	2728049	d.janevski@student.vu.nl
Diar Kamberi	2738305	d.kamberi@student.vu.nl
Kosta Bakalinov	2741568	k.bakalinov@student.vu.nl
Ibrahim Abdelraouf Ibrahim Badr	2737268	ibrahimabdelraoufibrabimbadr@student.vu.nl

Summary of changes from Assignment 2

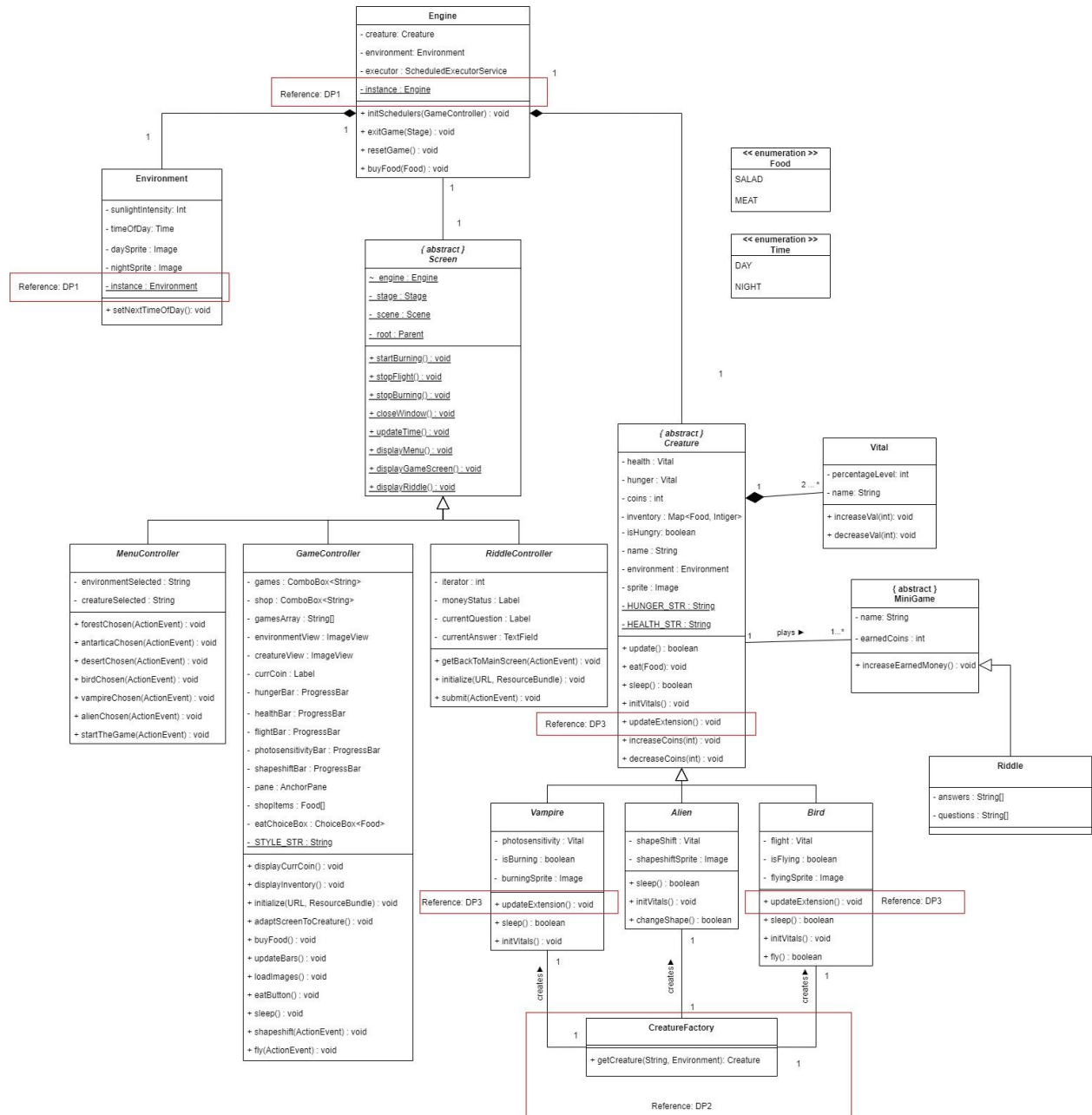
Author(s): Darian Janevski, Diar Kamberi, Kosta Bakalinov, and Ibrahim A. I. Badr

After receiving feedback from our mentor on Assignment 2, we further discussed the design of our project. The following list outlines all the changes we made to the UML diagrams of our project.

1. Class diagram:
 - Implementing the abstract screen class that is responsible for controlling the graphics of the game to keep the single responsibility of the engine, creature, and environment classes.
 - Removing the relationship of the enumerations.
 - Changing the relationship between the engine and the environment and the creature to composite instead of shared aggregation.
2. Object diagram:
 - Removing variables that were not present in the class diagram.
3. State machine diagram:
 - Renaming the states and activities that don't adhere to the UML guidelines.
4. Sequence diagram:
 - Specifying the return messages type for each sent message.
 - Adding a break to the game loop to indicate the creature's death.

Revised class diagram

Author(s): Darian Janevski, Diar Kamberi, c Kosta Bakalinov, and Ibrahim A. I. Badr



Link:

<https://app.diagrams.net/?src=about#G1csKtUUxNWEGOsYZDyGrYjQNrD9QsQIyU>

Description:

The class diagram depicted above models the class structure and relationships between the classes in our final “Vivarium” game design. From the previous class diagram, we have improved the classes we already had and we have included the classes with “technical purpose” which were part of the JavaFX library. The main changes in the classes we already had were that we removed the enumeration for **Color** since the color was handled by the scene-builder itself. Furthermore, we removed the positioning variables inside the **Creature** and **Environment** classes since the position of the sprites was also handled by the scene-builder. We also removed the **Shop** class since we realized that we could implement buying food without having another class, which would lower the coupling and the complexity of the program. The price of the items and logic is now handled internally in the buyFood() method in the **Engine**. The last class we removed was the **Initializer** class. This was done since we implemented all of the methods in the engine and we did not find a use for the **Initializer** as a separate class, effectively lowering the complexity of the program.

With the removal of several classes, we were able to implement the “technical purpose” classes that were used for controlling each screen and communicating with the engine. We added the **Screen** class to reduce cognitive load by reducing complexity. It is responsible for displaying the scenes on the screen and also keeps track of the main attributes needed by JavaFX which are the Parent, Stage, and Scene(which are JavaFX classes so we do not include them in our class diagram) instances that are shared between the subclasses. The **Screen** class has methods that allow for switching between the different screens (controllers should also change when the scene changes). The **Engine** now holds the schedulers and the methods exitGame(Stage) which closes the game and resetGame() which is self-explanatory. The **Engine** also initializes the schedulers which update the vitals and the time in the game with the function initSchedulers(GameController). From there we have the three different controllers used for the three different scenes, the menu scene, the main game scene, and the riddle scene.

The **MenuController** is used for the menu scene where the user chooses the environment and the creature. We have 8 different methods each for the button pressed on the screen and the corresponding choice. In the end, we have the method startTheGame(ActionEvent) which sets uses the chosen creature and environment and sends them to the **Engine** after which the **Engine** initializes the schedulers and changes the scene to the main game scene.

In the main game scene, the **GameController** is used. It is responsible for displaying all of the sprites and handling the changing in the sprites and the vitals. It also incorporates all of the buttons on the screen, their functionality, and the progress bars. The **GameController** is a large class since JavaFX requires the elements to be attributed in the controller if you want to do something with them. The amount of methods is also large since all of the buttons and visual updates are handled here. This is a deep class that provides essential functions of the GUI for gameplay.

The **RiddleController** is needed for the minigame scene to be able to incorporate JavaFX elements. It holds all of the functionality of the riddle minigame if you get a question right or wrong and changes the money earned. With the method goBackToMainScreen(), the vitals of the creature are updated and we go back to the main scene of the game.

Furthermore, we have created another class, the **CreatureFactory** which helps us use the Factory design pattern (DP3) for the creatures. If in the future there is a need for extending the game with more creatures, with this we add the new creature in the factory switch statement, and choosing the creature in the controller stays as clean code.

In the end, we kept the most important classes and only removed a few. Since the last adaptation of the class, we have implemented the logic behind the GUI and the classes needed for that. We also used several design patterns reflected in the class diagram.

Application of design patterns

Author(s): Darian Janevski, Diar Kamberi, Kosta Bakalinov, and Ibrahim A. I. Badr

	DP1
Design pattern	Singleton
Problem	Our engine class is quite important, handling most of the game logic. Our controller classes interact with the screen. Communication between the engine and the controllers is needed to ensure that visuals get updated and that button presses work. Thus, we needed global access to the engine class. Our engine class also needs to be the same instance throughout the code, otherwise, unexpected behavior might occur.
Solution	By utilizing the Singleton pattern, we now have global access to the engine instance, and we can ensure communication from the controllers to the engine.
Intended use	The Engine instance will be accessed by the controllers whenever a method from the Engine needs to be called or a value/attribute from the engine is needed. Such a case is instantiating the creature and the environment, where we needed to update the references of the Engine creature and environment to the chosen creature and environment. Another case is starting the schedulers, which are responsible for updating the creature vitals and environment time constantly when the start game button is pressed.
Constraints	The engine now provides a single point of failure. If the Engine singleton is somehow corrupted, the system will fail to function correctly. However, we do not expect this to happen in our relatively small project.

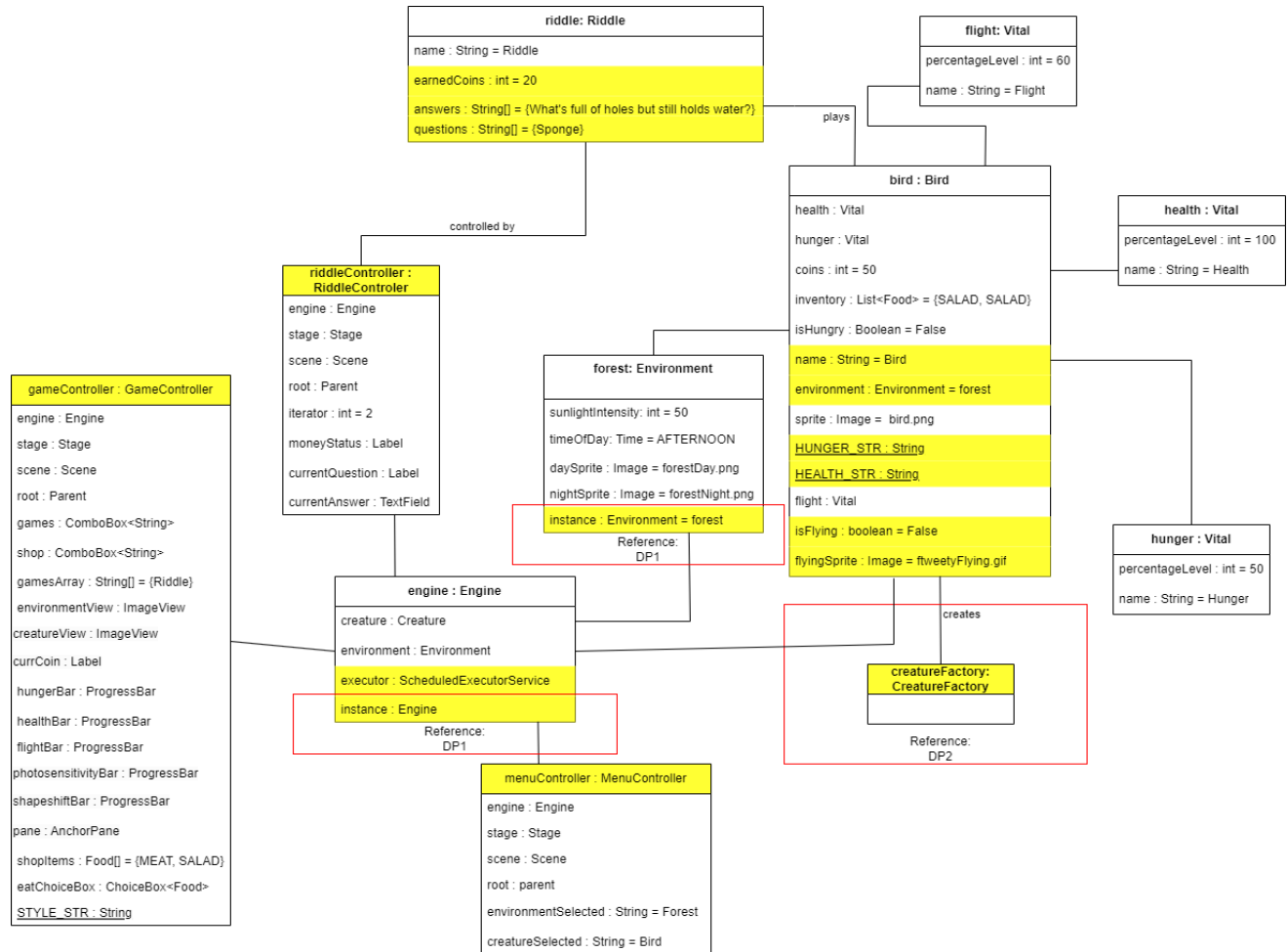
	DP2
Design	Factory

pattern	
Problem	Creating creatures was being done in an inefficient way. We had switch statements inside the controller, that would check what creature you chose based on the string that the button press passes to it.
Solution	The CreatureFactory can now be easily extended with new creatures, and our Controller class code for choosing a creature stays clean. Effectively decoupling the controller from creating the instances.
Intended use	With a CreatureFactory, we can now simply pass the string of the creature chosen to the factory, and we will get back a creature of that type. This means that any new creatures that are added to the game, need to be added to the factory in a case statement with the corresponding string.
Constraints	No actual constraints here, we just reduced the complexity of the system by removing the responsibility of the Controller to check the creature chosen.

	DP3
Design pattern	Template
Problem	We needed to reduce the amount of repeated code for the Bird and Vampire class and update the function.
Solution	Instead of overriding the update function, a subclass of Bird and Vampire can now add all the code needed to be checked when the update function is called to the updateExtension function.
Intended use	updateExtension function in Creature is abstract, so any class that inherits from it has to implement it. It is called first during the update function. That means that any code you have defined in it, will be executed, and then the update function will go on to update health and hunger vitals. In the case of the Alien, which does not have a vital that is updated continuously, the updateExtension is empty.
Constraints	The constraint here is that the hunger and health vitals will always be updated as normal after the updateExtension is executed. Meaning that if you want to add an animal that somehow does not need this behaviour, you would need to override the update function.

Revised object diagram

Author(s): Darian Janevski, Diar Kamberi, Kosta Bakalinov, and Ibrahim A. I. Badr



Link:

<https://app.diagrams.net/?src=about#G1flm3l5n6l9ATXDbD9FgOmEMhnTST2Y3e>

Description:

The object diagram above is compliant with our newly designed class diagram. All of the improvements and modifications made from the last object diagram are indicated in yellow color. This refers to either new attributes or new objects that didn't exist before because the corresponding classes were not invented yet. The red rectangles depict changes that reflect the usage of specific design patterns that we implemented to reduce the system's complexity.

Description:

The above diagram illustrates the state machine of the **Bird** class. Although the fundamental design remained the same as the previous assignment, some modifications were made to certain values and activities triggering the ***Sleeping***, ***Flying***, and ***Playing*** states.

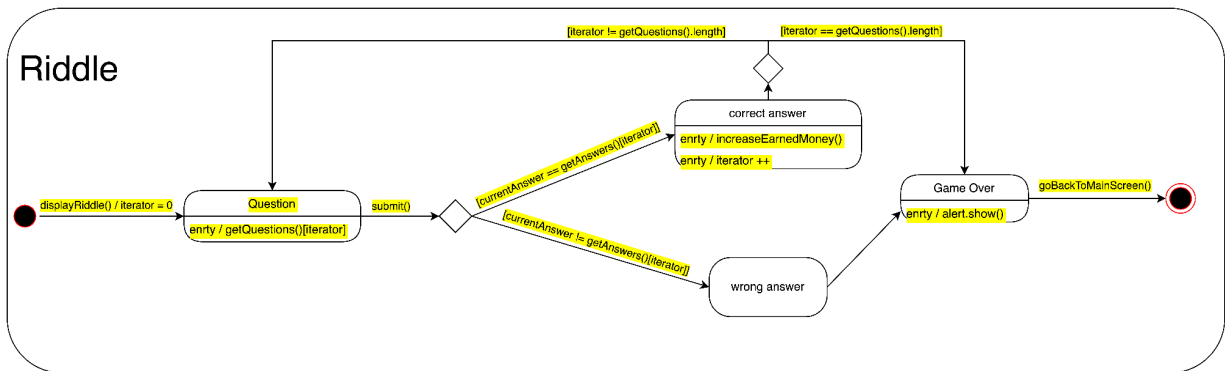
The enhancements include adjustments to the time at which the hunger level is reduced during the idle state, as well as the health level during the hunger state, both of which are set to five seconds for the demo video. Additionally, players can now fly the bird until the flight bar percentage reaches zero, or until they press the flying button again, compared to the previous model which only allowed for three seconds of flight time. These improvements are expected to make the game more engaging and enjoyable for young players.

Additionally, In our game design, we have eliminated the morning time since it doesn't affect the game design and now we only have day and night environments. Consequently, when the sleep() event is triggered, the bird will sleep only at night as in the previous model, and then return to idleness by calling the setTextTimeOfDay() function.

Furthermore, the *Riddle* mini-game is now triggered using the displayRiddle() event, which initializes a new game scene in JavaFX. Once the game is completed, the getEarnedMoney() function is called to add the earned coins to the player's Bird, and then the goBackToMainScreen() function is called to return to the main game. Unlike the previous design, which checked whether there were coins to be awarded or not, the getEarnedMoney() function now handles this situation, counting the number of earned coins and returning it, or returning zero coins if the first question is answered incorrectly.

In conclusion, these enhancements were implemented to make the game more appealing and exciting, while also following good software design practices such as encapsulation and simplicity of implementation. In the actual game design, values will be depleted at a slower rate.

Riddle class:



Link (for the two-state machine diagrams):

https://drive.google.com/file/d/1pTCV7i2G35DBCla5haOy5xaa_UenBMSH/view?usp=sharing

Description:

Similarly, the state machine diagram of the **Riddle** game class has undergone some minor improvements, including the implementation of new functions and the introduction of an **iterator** variable to keep track of the remaining questions in the questions' array.

Upon initialization of the game, the iterator value is set to zero, and the first question is displayed to the player. The submit() event then triggers the next state, which can either be the **correct answer** state if the player's response matches the answer in the answers array with the same index, or the **wrong answer** state, followed by the **game-over** state.

After the **correct answer** state exit, the iterator value is checked. If it is equal to the length of the questions array, then the **game-over** state is called. However, if the iterator value does not equal the length of the questions' array, the game immediately displays the next question.

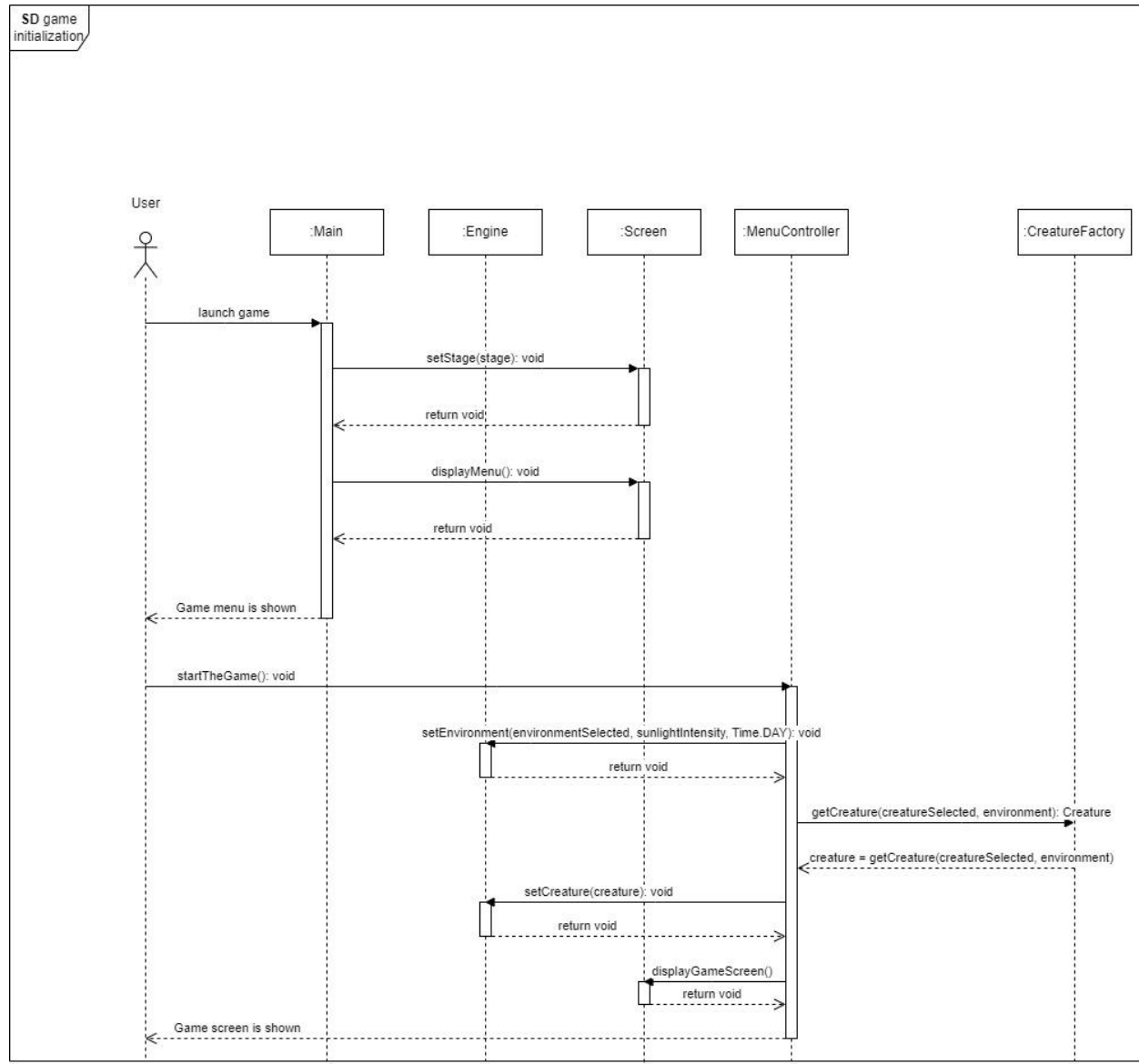
Another enhancement is implementing a new function, increaseEarnedMoney(), which is called only in case of a correct answer, rather than using a Boolean function to determine whether to award coins. This reduces the complexity of the code and function calls.

Finally, we have added an alert window at the **game-over** state that displays which vitals have been changed and the amount of earned coins. These improvements are expected to enhance the game experience and simplify the code implementation.

Revised sequence diagrams

Author(s): Darian Janevski, Diar Kamberi, Kosta Bakalinov, and Ibrahim A. I. Badr

Game Initialization:



Link(for the two sequence diagrams):

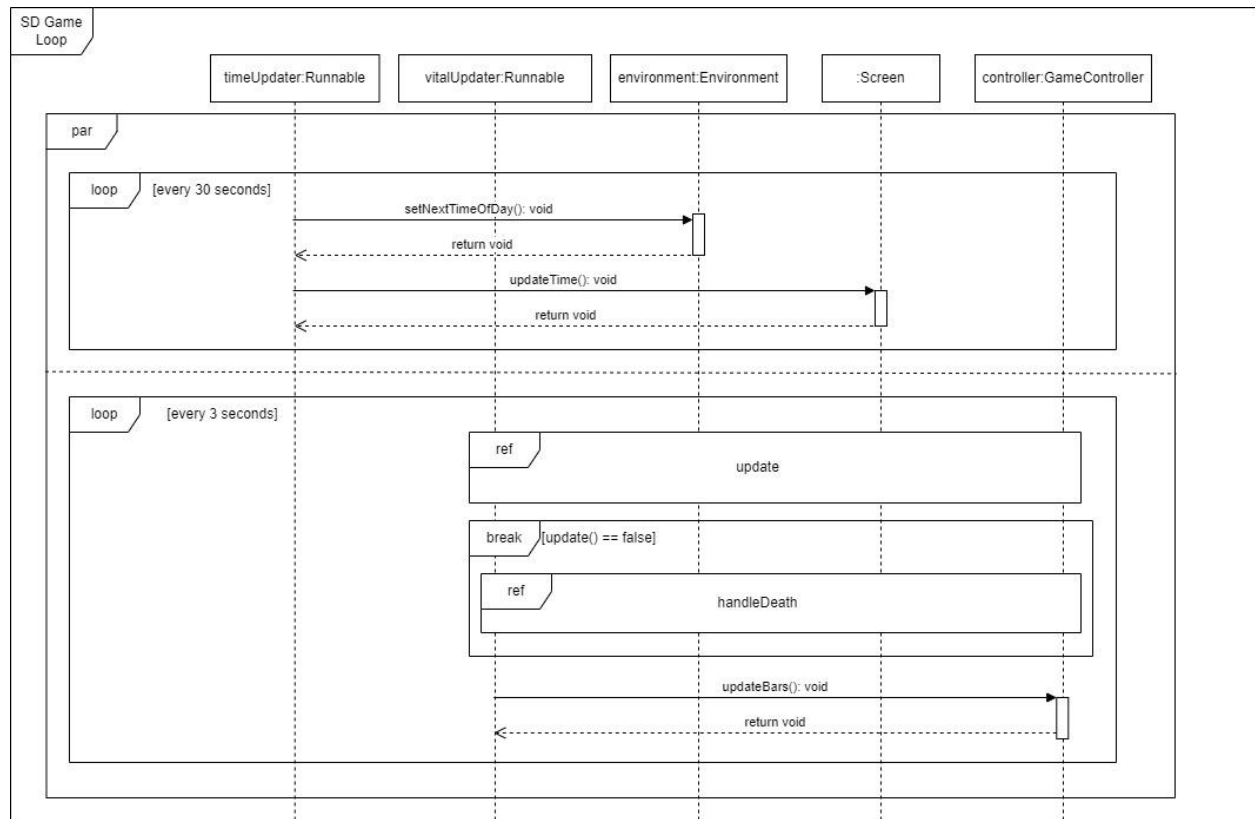
<https://drive.google.com/file/d/1dxBTZLJvZKnVoh9Y7GLTcF3SbSrh8vOv/view?usp=sharing>

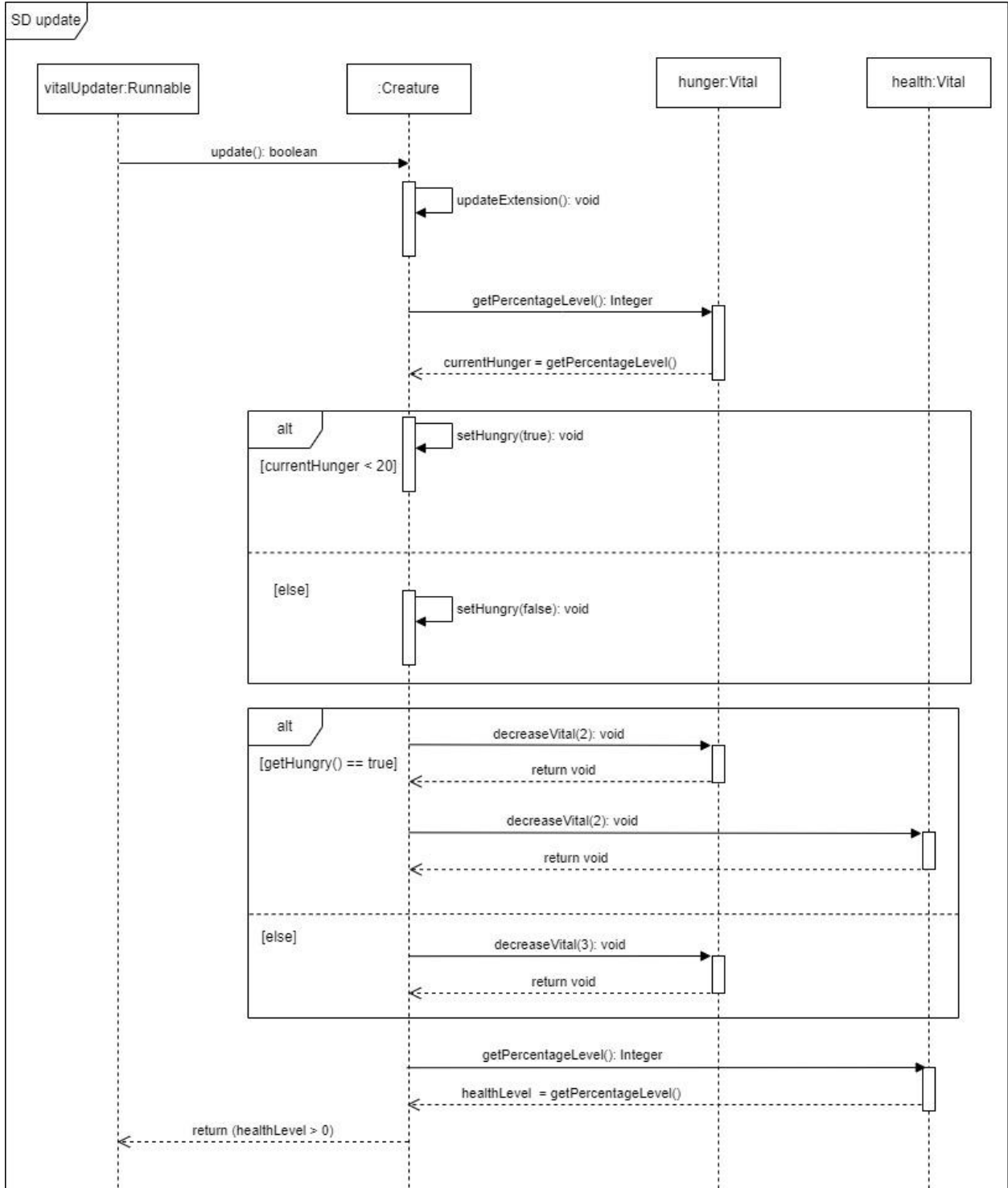
Description:

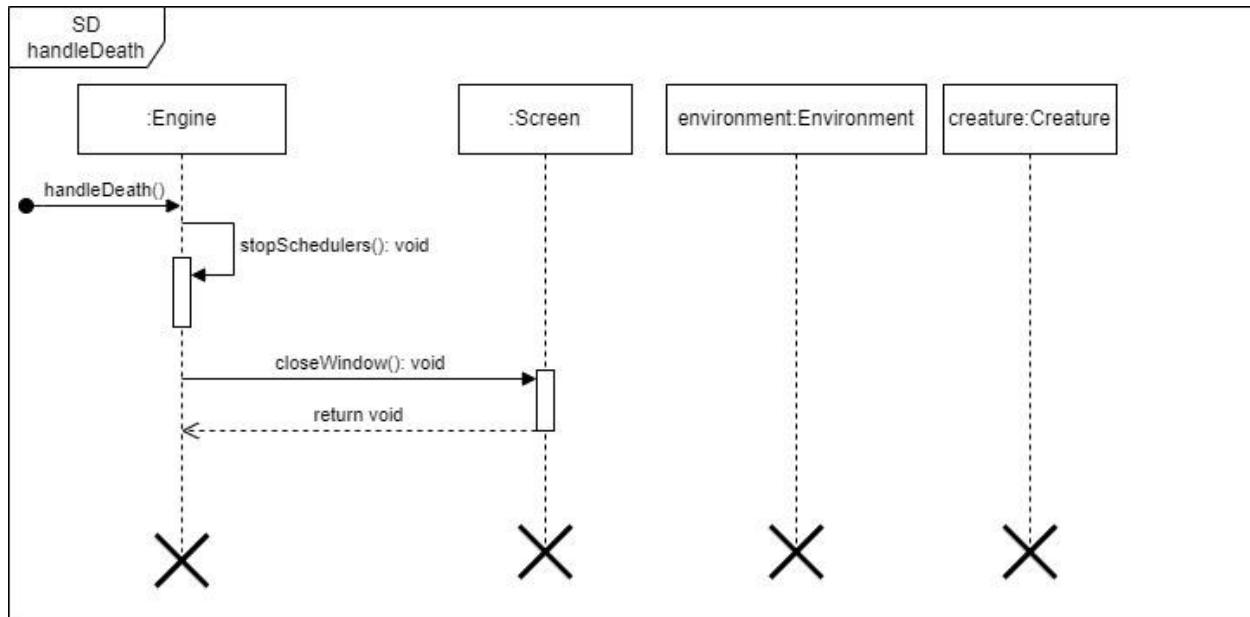
Note: We had to change the sequence diagrams quite a bit since we are doing the GUI. This is because we had no knowledge of how JavaFX interacts with objects during assignment 2. For this reason, we could not highlight the parts we changed in the sequence diagram because they are so different.

For the sequence diagram, we had to change some things due to the incorporation of GUI. We were not familiar with the specifics of JavaFX in assignment 2, hence, we changed stuff as we learned more about it in this assignment. Since we implemented a GUI, there have been a few noteworthy changes. First, the **Screen** class is responsible for displaying the initial scene and changing scenes and visuals. After the initial scene is shown, the **MenuController** is responsible for handling the start button press and the creature and environment choices. We used a factory design pattern to make the **Creature** as it provides a better division of tasks between controllers. Next, when the user starts the game, the **MenuController** is responsible for setting the creature and environment of the **Engine** to the chosen ones. After this is done, the displayGameScreen() is called to show the game screen to the user. With this, the game is initialized. The displayGameScreen() also initializes the schedulers, which we decided to abstract out of the diagram since most of this function handles JavaFX-specific functions that allow a scene to be shown. Overall, the game initialization has been simplified, with the addition of the controllers and the **Screen** class, which helped remove the responsibility of displaying graphics and handling the user interaction events out of the **Engine** class.

Game Loop:







Link(for the two sequence diagrams):

<https://drive.google.com/file/d/1dxBTZLJvZKnVoh9Y7GLTcF3SbSrh8vOv/view?usp=sharing>

Description:

Note: We had to change the sequence diagrams quite a bit since we are doing the GUI. This is because we had no knowledge of how JavaFX interacts with objects during assignment 2. For this reason, we could not highlight the parts we changed in the sequence diagram because they are so different.

The SD Game Loop is the name of the sequence diagram, it is not an actual loop. It represents the updates that happen in our game without user interaction which are namely: updating time and updating vitals. Both loops are run infinitely unless the schedulers are stopped externally. They are Runnable class objects that are handled from the Java ScheduledExecutorService.

The game loop remains conceptually similar to assignment 2, with some differences due to changes in our classes. The *vitalUpdater* and the *timeUpdater* are the instances of the scheduler functions that update the game without user interaction. The *timeUpdater* calls the setNextTimeOfDay() every 30 seconds. The addition here is that updateTime() is called and is responsible for showing the visual update to the screen. We used a template design pattern to limit the repeated code that would result from overriding the update() function. updateExtension() is now called first in the update() function, which is meant to handle any logic for different types of creatures(the function is abstract and must be implemented by any class that inherits from **Creature**). Now, any creature-specific updates would be added here by implementing the function. It can be empty and just return void in case it is not needed.

Next, we addressed the logical fallacies of our program, where we did not set the state of the *creature* back from being hungry if it ate food. Basically, we added an 'else' statement that sets isHungry to false if percentageLevel of *hunger* is above 20.

The game-over state is handled in the *vitalUpdater* lambda function, where we added a break statement to that loop (this break statement is only executed under the condition that update returns false). If hunger returns false, that means that the health level is 0 or lower, and hence the creature has died. A JavaFX specific is that the stage must be closed by the JavaFX Application thread, and since the schedulers run on different threads, we must make use of the Platform.runLater() function, which hands the handleDeath() task to the JavaFX Application thread. There was no easy fix to this and is a forced implementation decision and not a design decision. To illustrate that the caller is unknown or not relevant, we used a found message to the handleDeath() of the **Engine** that is only triggered when the creature's death conditions are fulfilled (health level is 0 or below).

In the handleDeath(), we stop the schedulers so that the program does not run in the background, and then we close the window by calling the closeWindow() on the **Screen**. After that the program stops execution and to play the game again, you have to launch the program again.

Implementation

Author(s): Darian Janevski, Diar Kamberi, Kosta Bakalinov, and Ibrahim A. I. Badr

Strategy to adopt UML to code

The phase of the designing of UML was used as a way to communicate the game requirements between the team members. After that was completed, we had to get acquainted with the JavaFX library, as we were creating a GUI for our software system. That was eventually going to change the UML diagrams.

As a start, we translated the class diagram entirely into code and we got a better feel of how our system will work. Then we had to figure out a way how to incorporate JavaFX controller classes with our current design in mind.

After completing the above-mentioned steps, we were really on the same page as a team and we were able to code independently on different software functionalities as there was already a layout of classes. Everyone could simply look up the diagrams for their parts and implement them into code accordingly. If they saw a better way to do it or an implication with the current UML design, they would reach out to all the team members to discuss it together. When we were coming to solutions, looking at the diagrams really helped us out.

Design decisions

Graphics/Animation

When it came to the game's appearance, the team's approach was to use images and gifs from the internet. The gifs were used for animations, such as using a flying bird clip when the **fly** button was triggered by the user. The decision was in favor of reducing the time and effort it would take if we were to design them ourselves.

Different controller classes for each game scene

Our game is currently comprised of three game screen files. The first screen is the game menu and the second screen is our main screen for the game where the user gets to interact with their creature. The third scene is the riddle game, thus the number of scenes will increase as the number of mini-games increases. We decided to have a separate controller for every scene for separation of concerns purposes. This way it was also easier to work together on the project. As someone was working on the controller responsible for scene 2, another teammate was working on the controller responsible for scene 3 i.e. the riddle game. No place for merge conflicts there.

The main reason for the different controllers was the fact that every scene would have a different layout, especially the mini-games.

Usability

We wanted to make the learning time of our interface very short. The user should look at the GUI and be aware of how everything works. This is very important for our main scene where the user interacts with the creature. The whole game functionality is therefore placed there.

- There are big vitals bars at the top right corner of the screen together with labels that show what they represent
- The top left corner has a shop drop-down menu with the label "Shop"
- Next to the shop is a spinning gif of a coin which displays the current money that the user possess
- Below the shop, there is a drop-down menu of mini-games with the label "Games"
- The bottom left has two buttons: the first one is the unique ability of the creature(if applicable) and the second is the sleep button. Both of them have labels
- The bottom right corner has a drop-down menu of food together with a button with the label "Eat"

The following is an image of the layout and interface in scene builder(3rd vital bar and a button above the "Sleep" button are missing because they are inserted dynamically, depending on what the user chooses)



Main class

The location for the main class is **com.example.softwaredesign.Main**

Jar file

As discussed with the TA, we have provided a not working Jar file. We couldn't link the dependencies and make a single executable jar file. The TA doesn't mind this and in addition, it doesn't affect the running of the game. The project can be run by running the command "gradle run" or by building the Gradle dependencies and running the main class.

Project Demo - https://youtu.be/2S1p_HqyI2Y

The YouTube video shows a small demo of our working game system. In the first part of the video, the whole functionality of the game can be seen from the perspective of the bird creature. The following things can be noted:

- Choosing between different creatures and environments in the game menu
- Buying and eating food
- Flying with the bird
- Playing a mini-game that earns you money based on your performance
- Sleep function(you can only sleep during the night with a bird creature)
- If the health percentage drops to zero, the creature dies and the game is over
- In the background the vitals keep decreasing and the user needs to deal with it or the creature dies

The second part of the video shows the unique trait of the vampire. It has photosensitivity vital, which if it drops below a certain threshold, causes the vampire to burn.

Finally, in the third part of the video, we can see the unique ability of the alien creature. It is the ability to change its shape while the corresponding vital is not critically low.

Time logs

Team Number:	22		
Member	Activity	Week number	Hours
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Discussing the feedback from assignment 2	5	3
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Split the task for assignment 3	5	1
-Diar Kamberi -Ibrahim A. I. Badr	Adding the JavaFX library to the project	5	4
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr"	Mentor meeting	5	1
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Discussing the feedback from the mentor meeting	5	3
-Darian Janevski	Implementing the classes layout in Java	6	3
-Kosta Bakalinov -Ibrahim A. I. Badr	Preparing the required resources (images and animations) for the GUI	6	4
-Ibrahim A. I. Badr	Implementing the game's main menu.	6	3
-Diar Kamberi	Implementing the game loop	6	4
-Kosta Bakalinov	Implementing the mini-game class and the riddle class	6	3

-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr"	Mentor meeting	6	1
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Discussing the feedback from the mentor meeting	6	2
-Darian Janevski	Implementing the sleeping functionality for all creatures	7	4
-Ibrahim A. I. Badr	Implementing the singleton design pattern for the Environment and Engine classes	7	3
-Diar Kamberi -Ibrahim A. I. Badr	Designing and displaying the bars in the GUI	7	4
-Kosta Bakalinov	Implementing the Riddle mini-game	7	4
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr"	Finalizing the game design and discussing implementing new design patterns	7	5
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr"	Mentor meeting	7	1
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Discussing the feedback from the mentor meeting	7	2
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Revising the UML diagrams	7	5
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Preparing the demo video	7	3
		TOTAL	63